Countability, Computability & the Halting Problem

Lecture

Outline

1. Same Size?
2. Countability
3. Rational Numbers
4. Bijections with $\mathbb{N}$
5. Cantor's Pairing Function
6. Algorithm
7. What About $\mathbb{R}$?
8. Computable Numbers
9. Computation & Undecidability
10. Halting Problem

Are the Following Sets of the Same Size?

Are the Following Sets of the Same Size?

For finite sets it is much easier to compare cardinality:

Key Question

How do we compare cardinality for *infinite* sets?

What Do We Understand by the Term Countability?

The elements of which of the following sets can you **count**?

$A = \{\text{Apple},\ \text{Orange},\ \text{Banana}\}$

$B = \{1,\ 2,\ 3,\ 4\}$

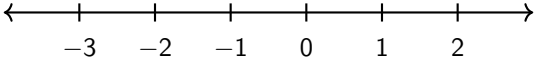
$\mathbb{N} = \{1,\ 2,\ 3,\ 4,\ \ldots\}$

$\mathbb{Z} = \{\ldots,\ -2,\ -1,\ 0,\ 1,\ 2,\ \ldots\}$

Countability — Definition

Can We Count Elements of Infinite Sets?

We have been listing the elements of $\mathbb{N}$ and $\mathbb{Z}$ in a linear fashion since forever. A number line!



What about $\mathbb{N}$?

Can we list the elements of $\mathbb{N}$?

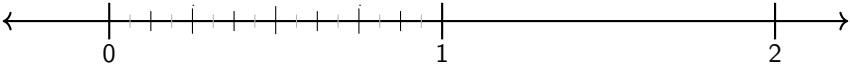
Yes! Cardinality? Infinite — makes sense!

What about $\mathbb{Z}$?

Can we list the elements of $\mathbb{Z}$?

Yes! Cardinality? Infinite — makes sense!

What About $\mathbb{Q}$? — Can We list the elements of $\mathbb{Q}$ the same way?



Rationals are **dense** — we cannot list each and every ration along a number line!

Cantor's Grid — Listing Each and Every Rationals in a Better Form

Traverse diagonals (skipping fractions not in lowest terms) $\Rightarrow$ every positive rational listed exactly once.

What Is Common About All These Sets?

Bijection: $\mathbb{Z} \to \mathbb{N}$

$$f(z) = \begin{cases} 2z & z > 0 \\ 1 & z = 0 \\ -2z + 1 & z < 0 \end{cases}$$

Cantor's Pairing Function

Definition

$$\pi(p,\, q) \;=\; \frac{(p+q-2)(p+q-1)}{2} \;+\; p$$

Using this, we define a bijection $W : \mathbb{Q} \to \mathbb{N}$:

$$W\left(\frac{p}{q}\right) = \begin{cases} 1 & p/q = 0 \\ 2\,\pi(p,\, q) & p/q = +\text{ve} \\ 2\,\pi(|p|,\, q) + 1 & p/q = -\text{ve} \end{cases}$$

where $\mathbb{Q} = \left\{\, p/q \;\middle|\; p,\, q \in \mathbb{Z},\; q \neq 0 \right\}$.

Bijection Between $\mathbb{Q}$ and $\mathbb{N}$

Cardinality & One-to-One Correspondence

Definitions

Let A and B be sets with cardinalities $|A| = m$ and $|B| = n$.

- **One-to-one (Injective):** $m \leq n$
  Each element in A maps to a distinct element in B
- **Onto (Surjective):** $m \geq n$
  Every element in B is mapped to by some element in A
- **One-to-one & Onto (Bijective):** $m = n$
  Perfect pairing between A and B

Equal Cardinality

Basically the infinities of $\mathbb{N}$, $\mathbb{Z}$ $\mathbb{Q}$, are equal. $\Rightarrow$ they have the **same cardinality**.

$$|\mathbb{N}| = |\mathbb{Z}| = |\mathbb{Q}| = \aleph_0$$

where $\aleph_0$ (aleph-naught) denotes the cardinality of countably infinite sets.

Algorithm: Print First 20 Rationals (Cantor's Grid)

```python
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def nth_rational(n):
    if n == 0:
        return (0, 1)
    count = 0
    diagonal = 1
    while True:
        for numerator in range(1, diagonal + 1):
            denominator = diagonal + 1 - numerator
            if gcd(numerator, denominator) == 1:
                count += 1
                if count == n:
                    return (numerator, denominator)
        diagonal += 1

for i in range(20):
    num, den = nth_rational(i)
    print(f"n={i}: {num}/{den} = {num/den:.4f}")
```

Dry Run

Diagonal d	n	Denominator	Fraction	Count

Extension: Printing Negative Rationals

```python
def nth_rational_all(n):
    if n == 0:
        return (0, 1)

    positive_index = (n + 1) // 2
    num, den = nth_rational(positive_index)

    if n % 2 == 1:
        num = -num

    return (num, den)

for i in range(20):
    num, den = nth_rational_all(i)
    print(f"n={i}: {num}/{den} = {num/den:.4f}")
```

Similarly, you will be able to print $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$.

What About The Set of Real Numbers? - Are They Countable Too?

Cantor's Diagonal Argument — Conclusion

Result

Hence, $(0, 1)$ is **uncountable**. Hence, $\mathbb{R}$ is **uncountable**.

Notation Alert: $|\mathbb{R}| = \mathfrak{c}$ (continuum)

$$\aleph_0 < \mathfrak{c} < 2^{\mathfrak{c}} < 2^{2^{\mathfrak{c}}} < \cdots$$

Countable Numbers vs. Computable Numbers

What Is Computation?

Computers as we know today grew out of a desire to **mechanise** all of mathematics and avoid bugs in mathematical reasoning — because humans can be wrong, biased towards their work and miss out certain things like Cantor did.

Hilbert's Programme (1900)

At the International Congress of Mathematicians, Hilbert set out the goal of **mechanising all of mathematics**.

Gödel & Turing (1930s)

Work by Gödel & Turing showed that such a program is **impossible**.

→ Gödel's Incompleteness Theorem

→ Undecidability of the Halting Problem

The Halting Problem

In 1936, Turing wanted to know if there was an algorithm that could look at any program and its input and decide:

"Will this ever finish?"

This is the **Halting Problem**. He discovered that this problem is **undecidable**.

Setup — Magic Predictor

Assume `WillItHalt(P, I)` exists:

- If P eventually stops $\Rightarrow$ returns `True` ("Halts")
- If P runs forever $\Rightarrow$ returns `False` ("Loops")

The `wrong` Program

Since we can always create a program `wrong(P)` which can take any program P as input:

```
def wrong(P):
    if WillItHalt(wrong, P) == True:
        while True:
            pass
    else:
        return
```

Now call: `wrong(wrong)`

`wrong(wrong)` — The Contradiction

`WillItHalt` **returns True**	`WillItHalt` **returns False**
`WillItHalt` says `wrong(wrong)` will **Halt**	`WillItHalt` says `wrong(wrong)` will **not Halt** (loop forever)
The `if` statement in `wrong` becomes True	The `if` statement in `wrong` becomes False
$\therefore$ enters while loop and it **never stops**	$\therefore$ enters else block and **returns** (stops)
The predictor said the program stops but it actually **loops**	The predictor said the program would loop forever but it **stops**
The predictor is wrong!	**The predictor is wrong again!**

Conclusion

In both cases the Magic Predictor gives a wrong answer. **No such** `WillItHalt` **can exist.** The Halting Problem is **undecidable**.

Thank You

Questions?